

QUESTION 2

Title

Case Study: Network Packet Analysis System (Sorting + Binary Search)

Aim:

To design and analyze a Network Packet Analysis System that efficiently processes large packet datasets using Sorting and Binary Search techniques.

Problem Statement:

Network monitoring systems generate large volumes of packet data. Efficient packet processing requires sorting packet records and quickly searching for specific packet IDs.

The system should:

- Sort packet data.
- Search packet information efficiently.
- Handle high-speed network traffic.
- Reduce processing delays.

Divide and Conquer Approach:

Sorting Stage

Use Quick Sort or Merge Sort.

Divide → Partition packets

Conquer → Sort partitions

Combine → Merge results

Time Complexity: $O(n \log n)$

Searching Stage

Use Binary Search on sorted packets.

Divide → Check middle packet

Conquer → Search relevant half

Time Complexity: $O(\log n)$

Key Issues:

Processing Speed

Real-time monitoring requires fast packet processing.

Large Data Volume

Millions of packets are generated continuously.

Scalability

System must support increasing traffic loads.:

Solution Design:

1. Capture packet records.
2. Sort packet IDs.
3. Apply Binary Search.
4. Return packet information.

Overall Time Complexity:

Sorting : $O(n \log n)$

Searching : $O(\log n)$

Total = $O(n \log n)$

Justification:

The solution is efficient because:

- Sorting organizes packets for rapid access.
- Binary Search minimizes lookup time.
- Divide and Conquer scales effectively for large datasets.
- Suitable for network monitoring environments.

Code:

```
def binary_search(arr, target):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

```
packets = [5005, 5001, 5009, 5003, 5008, 5002]
packets.sort()
target = 5008
result = binary_search(packets, target)
if result != -1:
    print("Packet Found at Index:", result)
else:
    print("Packet Not Found")
```

Sample Output:

Packet Found at Index: 4

Result:

The Network Packet Analysis System efficiently combines Sorting and Binary Search to process large packet datasets. The divide-and-conquer approach provides scalable performance, and the overall complexity of $O(n \log n)$ makes the solution suitable for real-time network monitoring systems.